

Boa, agora entendi o cenário completo. Você deslocou os vetores para `0x80000` (RAM), e o OrionCore já instalou handlers para bus error, address error, illegal instruction, div/0. Esses handlers provavelmente fazem o mínimo (imprimir algo e travar, ou reiniciar). O que você quer agora é **handlers melhores no kernel**, que imprimam informação útil (endereço do erro, PC, SR, registradores) em vez de só travar.

Vou te explicar **o que são essas exceções**, **o que o m68k empilha quando elas ocorrem**, e **como escrever handlers em C + assembly** para elas. Sem enrolação.

O que é uma exceção no 68000

Quando o 68000 encontra uma situação anormal (bus error, divisão por zero, instrução ilegal, etc.), ele:

1. **Salva o estado atual na pilha do supervisor (SSP)**:
 - Para a maioria das exceções: `SR` (2 bytes) + `PC` (4 bytes) = 6 bytes
 - Para **bus error** e **address error**: `SR` + `PC` + um **frame extra de 14 bytes** (endereço, instrução, etc.) = 20 bytes
2. **Lê o vetor** correspondente em `0x00000000`–`0x000003FF` (no seu caso, em `0x00080000`–`0x000803FF`, porque você deslocou)
3. **Salta** para o endereço que está no vetor
4. Continua a executar em **modo supervisor**, com interrupções mascaradas no nível da exceção

O handler da exceção é **só uma função** que roda nesse contexto. Quando ele termina, se quiser voltar, executa `rte` (Return from Exception), que restaura `SR` e `PC` da pilha.

Frame empilhado por exceção

Para a **maioria** das exceções (div/0, illegal, trap, privilege, etc.):

...

SP + 0: SR (2 bytes)
SP + 2: PC (4 bytes) <- endereço da instrução que causou a exceção
...

Para **bus error** e **address error**, o 68000 empilha mais coisa:

...

SP + 0: SR (2 bytes)
SP + 2: PC (4 bytes)
SP + 6: IR (2 bytes) <- instrução que causou
SP + 8: SR_extra (2 bytes) <- SR no momento do erro

```
SP + 10: addr_high (2 bytes)
SP + 12: addr_low (2 bytes) <- endereço acessado
SP + 14: ... (mais coisa, depende do modelo)
...`
```

Por isso o handler de bus error / address error precisa saber que o frame é maior.

O truque: um handler genérico em assembly + C

Em vez de escrever um handler em assembly para cada exceção, o padrão é:

1. ****Assembly****: salva todos os registradores na pilha, chama uma função C, e no final executa ``rte``.
2. ****C****: recebe um ponteiro para a pilha (com todos os registradores salvos) e imprime o que quiser.

Passo 1: Estrutura em C

```
`c
/* excecoes.h */
#ifndef EXCECOES_H
#define EXCECOES_H

#include <stdint.h>

/* Frame salvo pelo handler em assembly.
   A ordem DEVE casar com a ordem dos pushes no .S */
typedef struct {
    uint32_t d0, d1, d2, d3, d4, d5, d6, d7; /* D0-D7 */
    uint32_t a0, a1, a2, a3, a4, a5, a6; /* A0-A6 */
    uint32_t pc; /* PC no momento da exceção */
    uint16_t sr; /* SR no momento da exceção */
    uint16_t _pad; /* padding pra alinhar */
} ExcFrame;

void exc_bus_error(ExcFrame *f);
void exc_address_error(ExcFrame *f);
void exc_illegal(ExcFrame *f);
void exc_div0(ExcFrame *f);
void exc_priv(ExcFrame *f);
void exc_trapv(ExcFrame *f);
void exc_linea(ExcFrame *f);
void exc_linef(ExcFrame *f);
void exc_generic(ExcFrame *f, const char *nome);

#endif`
```

...

Passo 2: Handler genérico em C

```
```c
/* excecoes.c */
#include "excecoes.h"
#include <stdio.h> /* ou seu kprintf */

extern void kernel_puts(const char *s);
extern void kernel_puthex(uint32_t v); /* você precisa ter isso */
extern void kernel_putdec(uint32_t v);

static void dump_frame(ExcFrame *f) {
 kernel_puts("\n--- EXCECAO ---\n");
 kernel_puts("PC = "); kernel_puthex(f->pc); kernel_puts("\n");
 kernel_puts("SR = "); kernel_puthex(f->sr); kernel_puts("\n");
 kernel_puts("D0 = "); kernel_puthex(f->d0); kernel_puts("\n");
 kernel_puts("D1 = "); kernel_puthex(f->d1); kernel_puts("\n");
 kernel_puts("D2 = "); kernel_puthex(f->d2); kernel_puts("\n");
 kernel_puts("D3 = "); kernel_puthex(f->d3); kernel_puts("\n");
 kernel_puts("D4 = "); kernel_puthex(f->d4); kernel_puts("\n");
 kernel_puts("D5 = "); kernel_puthex(f->d5); kernel_puts("\n");
 kernel_puts("D6 = "); kernel_puthex(f->d6); kernel_puts("\n");
 kernel_puts("D7 = "); kernel_puthex(f->d7); kernel_puts("\n");
 kernel_puts("A0 = "); kernel_puthex(f->a0); kernel_puts("\n");
 kernel_puts("A1 = "); kernel_puthex(f->a1); kernel_puts("\n");
 kernel_puts("A2 = "); kernel_puthex(f->a2); kernel_puts("\n");
 kernel_puts("A3 = "); kernel_puthex(f->a3); kernel_puts("\n");
 kernel_puts("A4 = "); kernel_puthex(f->a4); kernel_puts("\n");
 kernel_puts("A5 = "); kernel_puthex(f->a5); kernel_puts("\n");
 kernel_puts("A6 = "); kernel_puthex(f->a6); kernel_puts("\n");
}

void exc_generic(ExcFrame *f, const char *nome) {
 kernel_puts("\n!!! EXCECAO: ");
 kernel_puts(nome);
 dump_frame(f);
 kernel_puts("\nSistema travado.\n");
 for (;;) {
 /* fica aqui pra sempre -- ou reinicia, ou volta pro scheduler */
 }
}

void exc_bus_error(ExcFrame *f) { exc_generic(f, "BUS ERROR"); }
void exc_address_error(ExcFrame *f) { exc_generic(f, "ADDRESS ERROR"); }
```

```

void exc_illegal(ExcFrame *f) { exc_generic(f, "ILLEGAL INSTRUCTION"); }
void exc_div0(ExcFrame *f) { exc_generic(f, "DIVISAO POR ZERO"); }
void exc_priv(ExcFrame *f) { exc_generic(f, "PRIVILEGE VIOLATION"); }
void exc_trapv(ExcFrame *f) { exc_generic(f, "TRAPV"); }
void exc_linea(ExcFrame *f) { exc_generic(f, "LINE A (nao implementado)"); }
void exc_linef(ExcFrame *f) { exc_generic(f, "LINE F (nao implementado)"); }
...

```

### ### Passo 3: Wrappers em assembly

Aqui está o pulo do gato. Você precisa de **um wrapper em assembly para cada exceção**, que:

1. Salva todos os registradores
2. Passa o `SP` como argumento (que aponta pro frame salvo)
3. Chama a função C
4. Restaura os registradores
5. Executa `rte`

```

```asm
/* excecoes_asm.S */

.text

/* Macro que gera um handler.
   \nome = nome da função C a chamar
   \label = label do handler */
.macro EXC_HANDLER label, cfunc
    .globl \label
\label:
    /* Salva D0-D7 */
    moveml %d0-%d7, %sp@-
    /* Salva A0-A6 */
    moveml %a0-%a6, %sp@-
    /* SP agora aponta pro frame salvo (ExcFrame) */
    moveal %sp, %a0      /* A0 = &frame */
    movel  %a0, %sp@-    /* empilha o argumento */
    jsr    \cfunc
    addql  #4, %sp      /* limpa o argumento */
    /* Restaura A0-A6 */
    moveml %sp@+, %a0-%a6
    /* Restaura D0-D7 */
    moveml %sp@+, %d0-%d7
    rte
.endm

```

```

EXC_HANDLER SvcBusError, exc_bus_error
EXC_HANDLER Svcaddress_err, exc_address_error
EXC_HANDLER SvcIllegalIns, exc_illegal
EXC_HANDLER SvcDiv0_handler, exc_div0
EXC_HANDLER SvcPrivViolation, exc_priv
EXC_HANDLER SvcTrapvIns, exc_trapv
EXC_HANDLER SvcLineA, exc_linea
EXC_HANDLER SvcLineF, exc_linef
...

```

****Atenção****: os nomes das funções C no seu vetor são `SvcBusError`, `Svcaddress\_err`, etc. Se você usar os mesmos nomes no assembly, o linker resolve. ****Mas**** no C eu usei `exc\_bus\_error`, etc. Ou você muda os nomes do vetor para casar com o C, ou faz um `#define`:

```

...c
/* No C, em vez de void exc_bus_error(...), use: */
void SvcBusError(ExcFrame *f) { exc_generic(f, "BUS ERROR"); }
void Svcaddress_err(ExcFrame *f) { exc_generic(f, "ADDRESS ERROR"); }
void SvcIllegalIns(ExcFrame *f) { exc_generic(f, "ILLEGAL"); }
void SvcDiv0_handler(ExcFrame *f) { exc_generic(f, "DIV0"); }
/* etc. */
...

```

Aí os nomes casam com os do vetor.

Passo 4: Instalar os handlers no vetor

Como você deslocou os vetores para `0x80000`, o vetor de bus error está em `0x80008`, address error em `0x8000C`, etc. Você já tem `sys\_setramvector`:

```

...c
sys_setramvector(0x00080008, (uint32_t)SvcBusError);
sys_setramvector(0x0008000C, (uint32_t)Svcaddress_err);
sys_setramvector(0x00080010, (uint32_t)SvcIllegalIns);
sys_setramvector(0x00080014, (uint32_t)SvcDiv0_handler);
sys_setramvector(0x00080020, (uint32_t)SvcPrivViolation);
sys_setramvector(0x0008001C, (uint32_t)SvcTrapvIns);
sys_setramvector(0x00080028, (uint32_t)SvcLineA);
sys_setramvector(0x0008002C, (uint32_t)SvcLineF);
...

```

****Ou****, se os endereços do vetor já apontam para os labels `SvcBusError`, etc., você não precisa fazer nada — o linker resolve. Depende de como o OrionCore montou o vetor. Se o OrionCore **** copia **** o vetor para `0x80000`, aí você precisa ****atualizar**** o vetor em `0x80000` com `sys\_setramvector`.

****Pelo seu comentário****, o vetor já está em ``0x80000`` e aponta para ``SvcBusError``, etc. Então ****os handlers já estão instalados**** — você só precisa ****implementá-los**** em C. Se os labels ``SvcBusError`` etc. forem definidos no seu kernel, o linker resolve e o vetor já aponta pra eles.

****Confirme****: onde estão definidos ``SvcBusError``, ``Svcaddress_err``, etc.? No OrionCore ou no kernel? Se for no OrionCore, você precisa ****sobrescrever**** o vetor com ``sys_setramvector``. Se for no kernel, o linker já resolve.

Passo 5: `kernel\_puthex` e `kernel\_putdec`

Você precisa de funções para imprimir em hexadecimal e decimal. Se não tem, aqui vão versões simples:

```
``c
void kernel_puthex(uint32_t v) {
    const char *hex = "0123456789ABCDEF";
    int i;
    int started = 0;
    kernel_puts("0x");
    for (i = 28; i >= 0; i -= 4) {
        int nib = (v >> i) & 0xF;
        if (nib != 0 || started || i == 0) {
            kernel_putchar(hex[nib]);
            started = 1;
        }
    }
}
```

```
void kernel_putdec(uint32_t v) {
    char buf[12];
    int i = 0;
    if (v == 0) { kernel_putchar('0'); return; }
    while (v > 0) {
        buf[i++] = '0' + (v % 10);
        v /= 10;
    }
    while (i > 0) kernel_putchar(buf[--i]);
}
...
```

Passo 6: Cuidado com o `rte`

O ``rte`` no final do wrapper ****restaura**** ``SR`` e ``PC`` da pilha, e volta para a instrução que causou a exceção. Se a exceção foi um ****bug real**** (acesso a

endereço inválido), o ``rte`` vai **executar a mesma instrução de novo** e gerar a exceção eternamente. Por isso, no ``exc_generic``, eu coloquei um ``for(;;)`` no final — o handler **nunca** retorna. É o comportamento correto para um erro fatal.

Se quiser que o handler **mate a task** em vez de travar o sistema, aí você precisa:

1. Não executar ``rte``
2. Chamar ``OS_TaskExit()`` ou similar
3. Deixar o scheduler escolher outra task

Isso é mais complexo. Para o primeiro momento, travar e imprimir o frame é suficiente.

Passo 7: Ordem dos registradores no frame

No assembly, a ordem dos ``moveml`` define a ordem dos campos na ``ExcFrame``. Eu usei:

```
``asm
moveml %d0-%d7, %sp@- /* empilha D0, D1, ..., D7 (D0 no topo) */
moveml %a0-%a6, %sp@- /* empilha A0, A1, ..., A6 (A0 no topo) */
``
```

E a struct em C:

```
``c
typedef struct {
    uint32_t d0, d1, ..., d7;
    uint32_t a0, a1, ..., a6;
    ...
} ExcFrame;
``
```

Cuidado: o ``moveml %d0-%d7, %sp@-`` empilha **D7 primeiro, D0 por último**? Ou **D0 primeiro**? No m68k, o ``movem`` com pré-decremento **empilha na ordem inversa**. Ou seja:

- ``moveml %d0-%d7, %sp@-`` → empilha D7, D6, ..., D0. No final, **D0 está no topo** (menor endereço).
- ``moveml %a0-%a6, %sp@-`` → empilha A6, A5, ..., A0. No final, **A0 está no topo**.

Então o frame fica (do menor endereço pro maior):

...

SP + 0: D0

```
SP + 4: D1
...
SP + 28: D7
SP + 32: A0
SP + 36: A1
...
SP + 56: A6
SP + 60: SR (empilhado pelo 68000)
SP + 62: PC (empilhado pelo 68000)
...
```

Mas espera — o 68000 empilha `SR` e `PC` ****primeiro****, antes do handler rodar. Então a pilha, quando o handler começa, é:

```
...
SP + 0: SR    (do 68000)
SP + 2: PC    (do 68000)
...
```

Depois o handler faz `moveml %d0-%d7, %sp@-`, e a pilha fica:

```
...
SP + 0: D0
SP + 4: D1
...
SP + 28: D7
SP + 32: SR    (do 68000)
SP + 34: PC    (do 68000)
...
```

Depois `moveml %a0-%a6, %sp@-`:

```
...
SP + 0: A0
SP + 4: A1
...
SP + 24: A6
SP + 28: D0
...
SP + 56: D7
SP + 60: SR
SP + 62: PC
...
```

Então a `ExcFrame` que o C vê (com `SP` como base) tem:

```

...
offset 0: A0
offset 4: A1
...
offset 24: A6
offset 28: D0
...
offset 56: D7
offset 60: SR (2 bytes)
offset 62: PC (4 bytes)
...

```

****Isso não casa com a struct que eu escrevi antes!**** A struct que eu escrevi tem D0 primeiro e A0 depois. Preciso corrigir.

Struct corrigida

```

```c
typedef struct {
 uint32_t a0, a1, a2, a3, a4, a5, a6; /* A0-A6 (offset 0-24) */
 uint32_t d0, d1, d2, d3, d4, d5, d6, d7; /* D0-D7 (offset 28-56) */
 uint16_t sr; /* SR (offset 60) */
 uint16_t _pad; /* padding (offset 62) */
 uint32_t pc; /* PC (offset 64) */
} ExcFrame;
```

```

E o `dump\_frame` usa `f->a0`, `f->d0`, `f->sr`, `f->pc` nessa ordem.

****Alternativa mais simples****: use `moveml %d0-%d7/%a0-%a6, %sp@-` em ****uma**** instrução. Aí a ordem fica:

```

```asm
moveml %d0-%d7/%a0-%a6, %sp@-
```

```

O m68k empilha na ordem inversa do mask: A6, A5, ..., A0, D7, D6, ..., D0. Então o topo é D0, e a struct fica:

```

...
offset 0: D0
offset 4: D1
...
offset 28: D7
offset 32: A0
...

```

```
offset 56: A6
offset 60: SR
offset 62: PC (não alinhado!)
...
```

O `PC` fica em offset 62, ****não alinhado a 4****. Isso é chato para acessar em C (o compilador gera `movel` em endereço ímpar → address error). Por isso, o melhor é ****alinhar**** o frame.

Solução limpa: alinhar o frame

```
``asm
.macro EXC_HANDLER label, cfunc
    .globl \label
\label:
    moveml %d0-%d7/%a0-%a6, %sp@- /* salva tudo */
    moveal %sp, %a0             /* A0 = &frame */
    movel %a0, %sp@-           /* argumento */
    jsr \cfunc
    addql #4, %sp
    moveml %sp@+, %d0-%d7/%a0-%a6 /* restaura tudo */
    rte
.endm
...
```

E a struct em C fica com ****ordem invertida**** (porque o `movem` empilha ao contrário):

```
``c
typedef struct {
    uint32_t d0, d1, d2, d3, d4, d5, d6, d7;
    uint32_t a0, a1, a2, a3, a4, a5, a6;
    uint16_t sr;
    uint16_t _pad;
    uint32_t pc;
} ExcFrame;
...
```

Mas o `PC` e o `SR` estão ****antes**** do `movem` na pilha (empilhados pelo 68000). Então, quando o handler começa, a pilha é:

```
...
SP + 0: SR
SP + 2: PC
...
```

Depois ``moveml %d0-%d7/%a0-%a6, %sp@-``, a pilha fica:

```
...
SP + 0: D0
...
SP + 28: D7
SP + 32: A0
...
SP + 56: A6
SP + 60: SR
SP + 62: PC
...
```

De novo, ``PC`` em offset 62, ímpar. ****Address error**** ao acessar ``f->pc`` em C.

Solução definitiva: empilhar um padding

Antes do ``movem``, empilhe 2 bytes de padding para alinhar:

```
``asm
\label:
    clrw  %sp@-          /* padding pra alinhar */
    moveml %d0-%d7/%a0-%a6, %sp@- /* salva tudo */
    moveal %sp, %a0
    movel  %a0, %sp@-
    jsr    \cfunc
    addql  #4, %sp
    moveml %sp@+, %d0-%d7/%a0-%a6
    addql  #2, %sp        /* remove padding */
    rte
...
```

Agora a pilha fica:

```
...
SP + 0: D0
...
SP + 28: D7
SP + 32: A0
...
SP + 56: A6
SP + 60: padding (2 bytes)
SP + 62: SR (2 bytes)
SP + 64: PC (4 bytes) <- alinhado!
...
```

E a struct:

```
```c
typedef struct {
 uint32_t d0, d1, d2, d3, d4, d5, d6, d7; /* 0-28 */
 uint32_t a0, a1, a2, a3, a4, a5, a6; /* 32-56 */
 uint16_t _pad; /* 60 */
 uint16_t sr; /* 62 */
 uint32_t pc; /* 64 -- alinhado! */
} ExcFrame;
```
```

****Agora sim**, `f->pc` está em offset 64 (múltiplo de 4), acessível em C sem address error.**

Resumo do que implementar

1. ****`excecoes.h`****: struct `ExcFrame` ****com padding**** e protótipos
2. ****`excecoes.c`****: `exc\_generic` + wrappers `SvcBusError`, `Svcaddress\_err`, etc.
3. ****`excecoes\_asm.S`****: macro `EXC\_HANDLER` gerando os 8 handlers
4. ****`kernel\_puthex` e `kernel\_putdec`****: se não tiver
5. ****Instalar****: via `sys\_setramvector` se os handlers forem do kernel; senão, o linker resolve
6. ****Testar****: force um div/0 no `kmain` e veja a mensagem

Teste

No `kmain`, adicione:

```
```c
volatile int a = 1, b = 0;
volatile int c = a / b; /* div/0 */
```
```

Deve imprimir:

```
...
!!! EXCECAO: DIVISAO POR ZERO
PC = 0x...
SR = 0x...
D0 = 0x...
...
Sistema travado.
...

```

Dica final

O `**`exc_generic` nunca retorna**` — ele fica em ``for(;;)``. Isso é intencional: se a exceção foi um bug real, tentar continuar só vai gerar mais exceções. Travar e imprimir o frame é o comportamento correto.

Quando você tiver syscalls e cada processo isolado, aí você pode evoluir para `**matar a task**` em vez de travar o sistema. Mas isso é depois.

Se quiser, me manda o código do OrionCore que instala os vetores (a parte que copia para ``0x80000``) que eu te digo se o ``sys_setramvector`` é necessário ou se o linker já resolve. E manda também a implementação atual dos ``SvcBusError``, etc., se já existir, que eu adapto.